



**MINI
MINES**

EXTRACTING WHAT MATTERS

MM OS · BLUEPRINT · AUGUST 2026

One sign-in in front of every company system.

ERPNext, Item Code Studio, ATT Platform, Twenty CRM and everything built next — reached from one place, with each person's access decided in one place instead of five, and a built-in way to ask for what does not exist yet.

74 EMPLOYEES

6 SERVICES AT LAUNCH

10 WEEKS TO V1

1 VPS, COOLIFY

VPN AND OFFICE NETWORK ONLY

THE PROBLEM

Five tools, five logins, and no single answer to who can do what

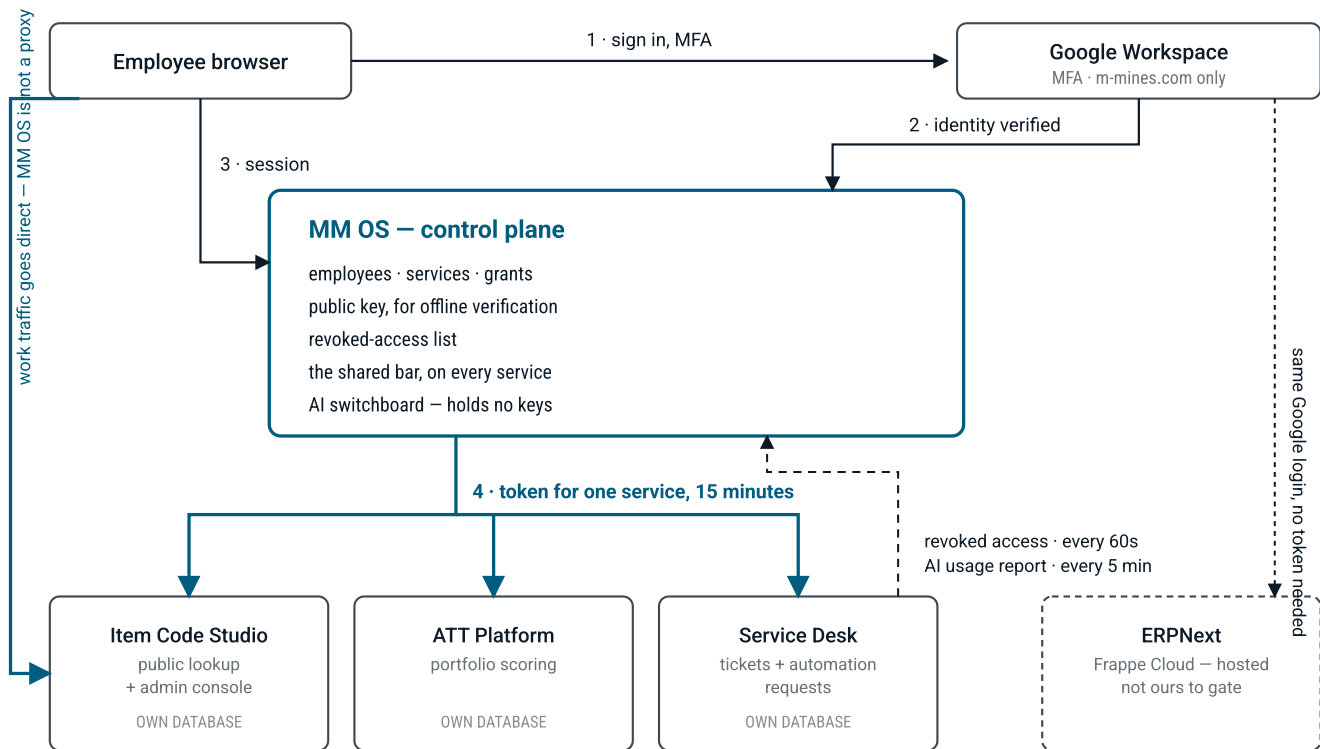
We keep building small services because they work — an item code generator, an analytics hub, a template engine. Each arrives with its own address, its own login or none at all, and its own idea of who is allowed in. Access lives in a spreadsheet, in someone's memory, and in a shared PIN. There is no place an employee can see what they have, and no place IT can go to change it.

MM OS fixes the layer underneath, not the tools. Every service keeps running as it does now, in its own container with its own database. What changes is that they all learn who you are from one source.

ERPNext is hosted software, so MM OS cannot own the traffic

ERPNext runs on Frappe Cloud. We do not control that server and cannot put our own middleware in front of it. So MM OS is not a gateway everything flows through — it is an **identity, directory and workflow layer**. It tells services who the visitor is; the services keep serving their own traffic directly.

That is also what makes it durable. Replace ERPNext or the CRM in two years and MM OS does not move.



The arrows in petrol are the ones that matter. MM OS hands out a token good for *one* service for fifteen minutes, and the actual work traffic never passes through MM OS — so if the control plane restarts, nobody inside a service is thrown out. Services check every 60 seconds for access that has been removed, and report their AI consumption every five minutes. ERPNext signs in against the same Google accounts without a token, because it owns its own session whatever we do.

What we chose, and what each choice costs

QUESTION	DECISION	WHY	WHAT IT COSTS
Who verifies people	Google Workspace, locked to m-mines.com, MFA enforced	Nobody manages another password, and offboarding is one click that closes every service at once	Staff without mailboxes need a code-and-PIN fallback
Who decides access	MM OS, as the master	Google has no idea what Item Code Studio is; access rules must live where the services are known	MM OS and ERPNext both hold employee records, so they can drift apart
How services trust us	A signed 15-minute token, checked without calling us, plus a 60-second revoked list	No network call on every request, so MM OS is never a single point of failure	Removing access takes up to 60 seconds rather than instantly
Permission shape	One row per person, per service, per role	Matches how the services actually work — Item Code Studio has viewers and admins, nothing more elaborate	Finer-grained rules stay inside each service
Front page	One page: logo, sign-in, and every service listed underneath	Anyone can see what exists and go straight to it; people who already have an account somewhere are not blocked	Service names are visible before signing in — accepted, since nothing is on the public internet
Existing accounts	ERPNext and Twenty keep their own sign-in; everything in-house goes through MM OS	Honest about what we control. Both routes use the same Google identity, so it is the same person either way	Two session owners to understand when something goes wrong
Feeling like one system	Services keep their own addresses; one shared bar sits on top of all of them	Nothing gets rewritten, and adding a service updates everyone's switcher with no redeployment	One script must be included in each service
Support and requests	A separate Service Desk application, wearing the MM OS shell	It serves every system including ERP and CRM, so it cannot live inside any one of them	One more service to run
AI keys	Stay inside each service. MM OS sees usage and holds the off switch	No central bundle of keys to leak, and cost is still visible in one place	Usage is only as accurate as what services report
Who can reach it	Office network and company VPN only	A stolen password is useless without network access; phishing and bot traffic stop mattering	Everyone needs a VPN profile, and the list must be audited monthly

No separate identity server. A dedicated product like Keycloak earns its place once two or more third-party applications demand it. Because our tokens use the standard format, adding one later is configuration rather than a rebuild.

THREE THINGS PEOPLE WILL NOTICE

Getting in, getting back, and asking for what does not exist

One page to get in

Logo, sign in with Google or an employee code and PIN, and every service listed underneath. Click one and it opens — in-house tools sign you in through MM OS on the way, ERPNext and Twenty use their own sign-in on the same Google account.

Always one click back

A slim bar sits at the top of every service: back to MM OS, a searchable switcher showing only what you can open, your open request count, and your role right here. ERPNext gets the same back-link through its own navbar settings — a supported setting, not a workaround.

Requests that end in something built

Anyone can request an automation. IT replies with a real proposal — scope, days, server and AI resources, and the cheaper alternative if one exists. The department head approves the cost. Then it gets built and appears in the list.

The alternative field is mandatory in every proposal. It is the cheapest way to stop a department commissioning a service that a saved report would have solved, and it turns the manager's approval into a real choice rather than a signature.

The approver is worked out from the reporting line, never chosen by the requester, and what was approved is frozen so later growth in scope is visible. Everyone in a department can see the requests it raised and who is handling them; a request can be marked private, which keeps its detail between the requester, whoever is handling it and the approver.

What ships, and what deliberately waits

IN V1

- Google sign-in, plus code and PIN for shop-floor staff
- All 74 employees, imported from the existing sheet
- One entry page listing every service, filtered to your access once you are in
- Access administration: who holds what, what each role permits, expiry dates, full history, and access requests decided in place
- The shared bar and switcher across every service
- ATT Platform and Item Code Studio moved across, their old PIN gates deleted
- Service Desk: my requests, the department queue, the IT console and approvals
- AI usage view with a per-service off switch
- A complete record of who granted what, to whom, when

WAITING FOR V2

- Unified mail inbox inside MM OS
- Search across every service at once
- Response-time targets and escalation on requests
- Raising requests by email
- AI cost dashboards and budgets
- Drift report against ERPNext employee records

The inbox and email intake wait together on purpose, so they are designed once. The service registry built in v1 is what makes cross-service search possible later.

Nine agents, two runs

Every contract — the database, every endpoint, the token format, the integration rules — is written before any feature code. That is what lets six agents build at the same time without waiting on each other: none has to guess what another will produce, and no two ever open the same file.

Run one · six in parallel

A1 – A6

- Identity, people, spreadsheet import
- Tokens, registry, AI switchboard
- The console people actually see
- The client library and shared bar
- Service Desk, end to end
- Deployment, VPN, backups, runbooks

Run two · three agents

B1 – B3

- Assemble the six branches, prove it works end to end
- Move ATT and Item Code Studio across
- Security review, runbook, floor guide, cutover

Roughly ten weeks

T0 V1

- The parts that slip are DNS, certificates and the Google console — so they go first
- Every agent finishes by writing down what it assumed and what it left undone
- One script proves the eight release criteria, and keeps proving them afterwards

Open item. Three console directions are built and waiting on a choice — a left rail with dense rows, a top navigation with a calmer layout, or a command-driven console with almost no chrome. Everything else in this document is settled.

What we are carrying on purpose

RISK	HOW IT IS HANDLED
Removing access takes up to 60 seconds	Accepted. Deactivating someone also ends their sessions immediately and stops new tokens at once, and disabling their Google account closes sign-in entirely.
Item Code Studio serves the public and admins from one application	Three structural locks rather than discipline: every write sits behind one guard; the public list is an allowlist, so a forgotten page fails closed; and the public side reads through a database account with no write permission at all.
Service names are visible before signing in	Accepted deliberately, so people can go straight to a tool they already use. Nothing is on the public internet, and that page carries names and addresses only — no roles, no people, no status.
Employee records exist in both MM OS and ERPNext	MM OS is authoritative for access. A drift report is scheduled for v2 rather than pretended away.
AI usage depends on services reporting it	A service that does not report shows as UNREPORTED — visibly, not as a blank row.
A private network is friction, and people route around friction	The real adoption risk. Watched for, and answered with usability rather than policy memos.
VPN access lists rot	Monthly audit of profiles against active employees, as a scheduled task with a command to run — not someone's memory.

Backups are only real once restored. A restore into a scratch database is part of the release criteria and repeats quarterly. An untested backup is a rumour.